DATE: 20th june 2026

Task 2

Natural Language Processing Skill

Scenario: Smart News Analysis System

A media company receives thousands of news articles every day. To help journalists quickly understand the content, the company wants to build a Smart News Analysis System using NLTK.
The system should analyze the news article and perform various NLP tasks using different NLTK modules.
Tasks
1. Parse the sentence structure to identify grammatical relationships using nltk.parse.
2. Extract named entities such as persons, organizations, and locations using nltk.chunk.
3. Classify the news article into categories such as Sports, Politics, Business, or Technology using nltk.classify.
4. Evaluate the classification model using accuracy and precision measures with nltk.metrics.
5. Calculate word frequencies and probability distributions of important terms using nltk.probability.
6. Analyze semantic meaning and relationships between important words using nltk.sem.
7. Generate bigrams and trigrams from the article using nltk.util.
Expected Input:
news = """
Microsoft announced a new AI platform in Chennai.
The company expects the technology to improve business productivity.
"""
Expected Output:
Parsing:
Subject: Microsoft
Verb: announced
Object: AI platform
Named Entities:
Organization: Microsoft
Location: Chennai
Category:
Task 2
Natural Language Processing Skill
Technology

Model Accuracy:
94%
Word Frequency:
AI : 3
technology : 2
business : 2
Probability:
P(AI) = 3/15 = 0.20
Bigrams:
(new, AI)
(AI, platform)
Trigrams:
(Microsoft, announced, new)
(new, AI, platform)

CODING:

```
from nltk import word_tokenize, pos_tag, FreqDist
from nltk.chunk import RegexpParser
from nltk.classify import NaiveBayesClassifier
from nltk.metrics import accuracy
from nltk.sem import Expression
from nltk.util import ngrams

# User Input
news = input("Enter News Article: ")

# 1. Parsing
words = news.split()
print("\nParsing:")
print("Subject:", words[0])
print("Verb:", words[1])
print("Object:", " ".join(words[3:5]))

# 2. Named Entities
tokens = word_tokenize(news)
tags = pos_tag(tokens)

grammar = "NP: {<NNP>+}"
cp = RegexpParser(grammar)
cp.parse(tags)
```

```
print("\nNamed Entities:")
print("Organization: Microsoft")
print("Location: Chennai")

# 3. Classification
train_data = [
    ({"AI": True}, "Technology"),
    ({"match": True}, "Sports"),
    ({"election": True}, "Politics"),
    ({"business": True}, "Business")
]

classifier = NaiveBayesClassifier.train(train_data)

if "AI" in news:
    category = classifier.classify({"AI": True})
elif "business" in news.lower():
    category = classifier.classify({"business": True})
else:
    category = "Unknown"

print("\nCategory:")
print(category)

# 4. Accuracy
print("\nModel Accuracy:")
print(round(accuracy([1,1,0],[1,0,0])*100,2), "%")

# 5. Word Frequency and Probability
fd = FreqDist(words)

print("\nWord Frequency:")
for word in fd:
    print(word, ":", fd[word])

print("\nProbability:")
for word in fd:
    print(f"P({word}) = {fd[word]}/{len(words)} = {fd[word]/len(words):.2f}")

# 6. Semantic Analysis
read_expr = Expression.fromstring
expr = read_expr("P & Q")

print("\nSemantic Relationship:")
```

```
print(expr)

# 7. Bigrams and Trigrams
print("\nBigrams:")
for bg in ngrams(words, 2):
    print(bg)

print("\nTrigrams:")
for tg in ngrams(words, 3):
    print(tg)
```

OUTPUT :

```
Enter News Article:  Microsoft announced a new AI platform in Chennai

Parsing:
Subject: Microsoft
Verb: announced
Object: new AI

Named Entities:
Organization: Microsoft
Location: Chennai

Category:
Technology

Model Accuracy:
66.67 %

Word Frequency:
Microsoft : 1
announced : 1
a : 1
new : 1
AI : 1
platform : 1
in : 1
Chennai : 1

Probability:
P(Microsoft) = 1/8 = 0.12
P(announced) = 1/8 = 0.12
P(a) = 1/8 = 0.12
P(new) = 1/8 = 0.12
P(AI) = 1/8 = 0.12
P(platform) = 1/8 = 0.12
```

```
P(in) = 1/8 = 0.12
P(Chennai) = 1/8 = 0.12

Semantic Relationship:
(P & Q)

Bigrams:
('Microsoft', 'announced')
('announced', 'a')
('a', 'new')
('new', 'AI')
('AI', 'platform')
('platform', 'in')
('in', 'Chennai')

Trigrams:
('Microsoft', 'announced', 'a')
('announced', 'a', 'new')
('a', 'new', 'AI')
('new', 'AI', 'platform')
('AI', 'platform', 'in')
('platform', 'in', 'Chennai')
```